JavaScript is disabled on your browser.

Java SE 27 & JDK 27

Class Runtime

java.lang.Object
    java.lang.Runtime

```
public final class Runtime
extends Object
```

Every Java application has a single instance of class `Runtime` that allows the application to interface with the environment in which the application is running. The current runtime can be obtained from the `getRuntime` method.

An application cannot create its own instance of this class.

Shutdown Sequence

The Java Virtual Machine initiates the *shutdown sequence* in response to one of several events:

1. when the number of live non-daemon threads drops to zero for the first time (see note below on the JNI Invocation API);
2. when the `Runtime.exit` or `System.exit` method is called for the first time; or
3. when some external event occurs, such as an interrupt or a signal is received from the operating system.

At
the beginning of the shutdown sequence, the registered shutdown hooks are started in some unspecified order. They run concurrently with any daemon or non-daemon threads that were alive at the beginning of the shutdown sequence.

After the shutdown sequence has begun, registration and de-registration of shutdown hooks with `addShutdownHook` and `removeShutdownHook` is prohibited. However, creating and starting new threads is permitted. New threads run concurrently with the registered shutdown hooks and with any daemon or non-daemon threads that are already running.

The shutdown sequence finishes when all shutdown hooks have terminated. At this point, the Java Virtual Machine terminates as described below.

It is possible that one or more shutdown hooks do not terminate, for example, because of an infinite loop. In this case, the shutdown sequence will never finish. Other threads and shutdown hooks continue to run and can terminate the JVM via the `halt` method.

Prior to the beginning of the shutdown sequence, it is possible for a program to start a shutdown hook by calling its `start` method explicitly. If this occurs, the behavior of the shutdown sequence is unspecified.

Java Virtual Machine Termination

The JVM terminates when the shutdown sequence finishes or when `halt` is called. In contrast to `exit`, the `halt` method does not initiate the shutdown sequence.

When the JVM terminates, all threads are immediately prevented from executing any further Java code. This includes shutdown hooks as well as daemon and non-daemon threads. This means, for example, that:

- threads' current methods do not complete normally or abruptly;
- `finally` clauses are not executed;
- uncaught exception handlers are not run; and
- resources opened with try-with-resources are not closed;

Implementation Note:

Native code typically uses the JNI Invocation API to control launching and termination of the JVM. Such native code invokes the `JNI_CreateJavaVM` function to launch the JVM. Subsequently, the native code invokes the `DestroyJavaVM` function to await termination of that JVM. The `DestroyJavaVM` function is responsible for initiating the shutdown sequence when the number of live non-daemon threads first drops to zero. When the shutdown sequence completes and the JVM terminates, control is returned to the native code that invoked `DestroyJavaVM`. This behavior differs from the `exit` or `halt` methods. These methods typically terminate the OS process hosting the JVM and do not interact with the JNI Invocation API.

See *Java Language Specification*:

12.8 Program Exit

Since:

1.0

External Specifications

Java Native Interface Specification

See Also:

`getRuntime()`

Nested Class Summary

Nested Classes

Modifier and Type	Class	Description

static final class	Runtime.Version	A representation of a version string for an implementation of the Java SE Platform.

Method Summary

All Methods | Static Methods

Modifier and Type	Method	Description
void	addShutdownHook(Thread hook)	Registers a new virtual-machine shutdown hook.
int	availableProcessors()	Returns the number of processors available to the Java virtual machine.
Process	exec(String command)	**Deprecated.** This method is error-prone and should not be used, the corresponding method exec(String[]) or ProcessBuilder should be used instead.
Process	exec(String[] cmdarray)	Executes the specified command and arguments in a separate process.
Process	exec(String[] cmdarray, String[] envp)	Executes the specified command and arguments in a separate process with the specified environment.
Process	exec(String[] cmdarray, String[] envp, File dir)	Executes the specified command and arguments in a separate process with the specified environment and working directory.
Process	exec(String command, String[] envp)	**Deprecated.**

		This method is error-prone and should not be used, the corresponding method `exec(String[], String[])` or `ProcessBuilder` should be used instead.
`Process`	**`exec`**`(String command, String[] envp, File dir)`	**Deprecated.** This method is error-prone and should not be used, the corresponding method `exec(String[], String[], File)` or `ProcessBuilder` should be used instead.
`void`	**`exit`**`(int status)`	Initiates the shutdown sequence of the Java Virtual Machine.
`long`	**`freeMemory`**`()`	Returns the amount of free memory in the Java Virtual Machine.
`void`	**`gc`**`()`	Runs the garbage collector in the Java Virtual Machine.
`static Runtime`	**`getRuntime`**`()`	Returns the runtime object associated with the current Java application.
`void`	**`halt`**`(int status)`	Immediately terminates the Java Virtual Machine.
`void`	**`load`**`(String filename)`	**Restricted.** Loads the native library specified by the filename argument.
`void`	**`loadLibrary`**`(String libname)`	**Restricted.**

		Loads the native library specified by the `libname` argument.
`long`	**`maxMemory`**`()`	Returns the maximum amount of memory that the Java virtual machine will attempt to use.
`boolean`	**`removeShutdownHook`**`(Thread hook)`	De-registers a previously-registered virtual-machine shutdown hook.
`void`	**`runFinalization`**`()`	**Deprecated, for removal: This API element is subject to removal in a future version.** Finalization has been deprecated for removal.
`long`	**`totalMemory`**`()`	Returns the total amount of memory in the Java virtual machine.
`static Runtime.Version`	**`version`**`()`	Returns the version of the Java Runtime Environment as a `Runtime.Version`.

Methods declared in class Object

`clone`, `equals`, `finalize`, `getClass`, `hashCode`, `notify`, `notifyAll`, `toString`, `wait`, `wait`, `wait`

Method Details

getRuntime

```
public static Runtime getRuntime()
```

Returns the runtime object associated with the current Java application. Most of the methods of class `Runtime` are instance methods and must be invoked with respect to the current runtime object.

Returns:

the `Runtime` object associated with the current Java application.

exit

```
public void exit(int status)
```

Initiates the shutdown sequence of the Java Virtual Machine. This method initiates the shutdown sequence (if it is not already initiated) and then blocks indefinitely. This method neither returns nor throws an exception; that is, it does not complete either normally or abruptly.

The argument serves as a status code. By convention, a nonzero status code indicates abnormal termination.

Successful invocations of this method are serialized such that only one invocation initiates the shutdown sequence and terminates the VM with the given status code. All other invocations will perform no action and block indefinitely.

Because a successful invocation of this method blocks indefinitely, if it is invoked from a shutdown hook, it will prevent that shutdown hook from terminating. Consequently, this will prevent the shutdown sequence from finishing.

The `System.exit` method is the conventional and convenient means of invoking this method.

Implementation Note:

If the system logger for `java.lang.Runtime` is enabled with logging level `Level.DEBUG` the stack trace of the call to `Runtime.exit()` is logged.

Parameters:

`status` - Termination status. By convention, a nonzero status code indicates abnormal termination.

See Also:

`addShutdownHook(Thread)`, `removeShutdownHook(Thread)`, `halt(int)`

addShutdownHook

```
public void addShutdownHook(Thread hook)
```

Registers a new virtual-machine shutdown hook.

A *shutdown hook* is simply an initialized but unstarted thread. Shutdown hooks are started at the beginning of the shutdown sequence. Registration and de-registration of shutdown hooks is disallowed once the shutdown sequence has begun.

Uncaught exceptions are handled in shutdown hooks just as in any other thread, as specified in `Thread.UncaughtExceptionHandler`. After the uncaught exception handler has completed, the shutdown hook is considered to have terminated and is not treated differently from a hook that has terminated without having thrown an uncaught exception.

API Note:

Shutdown hooks run at a delicate time in the life cycle of a virtual machine and should therefore be coded defensively. They should, in particular, be written to be thread-safe and to avoid deadlocks insofar as possible. They should also not rely blindly upon services that may have registered their own shutdown hooks and therefore may themselves be in the process of shutting down. Attempts to use other thread-based services such as the AWT event-dispatch thread, for example, may lead to deadlocks.

Shutdown hooks should also finish their work quickly. When a program invokes `exit`, the expectation is that the virtual machine will promptly shut down and exit. When the virtual machine is terminated due to user logoff or system shutdown the underlying operating system may only allow a limited amount of time in which to shut down and exit. It is therefore inadvisable to attempt any user interaction or to perform a long-running computation in a shutdown hook.

Parameters:

`hook` - An initialized but unstarted `Thread` object

Throws:

`IllegalArgumentException` - If the same hook (compared using ==) as the specified hook has already been registered, or if it can be determined that the hook is already running or has already been run

`IllegalStateException` - If the shutdown sequence has already begun

Since:

1.3

See Also:

`removeShutdownHook(Thread)`, `halt(int)`, `exit(int)`

removeShutdownHook

```
public boolean removeShutdownHook(Thread hook)
```

De-registers a previously-registered virtual-machine shutdown hook. Hooks are compared using ==. Registration and de-registration of shutdown hooks is disallowed once the shutdown sequence has begun.

Parameters:

`hook` - the hook to remove

Returns:

`true` if the specified hook had previously been registered and was successfully de-registered, `false` otherwise.

Throws:

`IllegalStateException` - If the shutdown sequence has already begun

Since:

1.3

See Also:

`addShutdownHook(Thread)`, `exit(int)`

halt

```
public void halt(int status)
```

Immediately terminates the Java Virtual Machine. Termination of the Java Virtual Machine is unconditional and immediate. This method does not initiate the shutdown sequence, nor does it wait for the shutdown sequence to finish if it is already in progress. An invocation of this method never returns normally.

API Note:

This method should be used with extreme caution. Using it may circumvent or disrupt any cleanup actions intended to be performed by shutdown hooks, possibly leading to data corruption. See the termination section above for other possible consequences of halting the Java Virtual Machine.

Parameters:

`status` - Termination status. By convention, a nonzero status code indicates abnormal termination. If the `exit` (equivalently, `System.exit`) method has already been invoked then this status code will override the status code passed to that method.

Since:

1.3

See Also:

`exit(int)`, `addShutdownHook(Thread)`, `removeShutdownHook(Thread)`

exec

```
@Deprecated(since="18")
public Process exec(String command)
            throws IOException
```

Deprecated.
This method is error-prone and should not be used, the corresponding method `exec(String[])` or `ProcessBuilder` should be used instead. The command string is broken into tokens using only whitespace characters. For an argument with an embedded space, such as a filename, this can cause problems as the token does not include the full filename.

Executes the specified string command in a separate process.

This is a convenience method. An invocation of the form `exec(command)` behaves in exactly the same way as the invocation `exec(command, null, null)`.

Implementation Note:

In the reference implementation, logging of the created process can be enabled, see `ProcessBuilder.start()` for details.

Parameters:

`command` - a specified system command.

Returns:

A new `Process` object for managing the subprocess

Throws:

`IOException` - If an I/O error occurs

`NullPointerException` - If `command` is `null`

`IllegalArgumentException` - If `command` is empty

See Also:

`exec(String[], String[], File)`,
`ProcessBuilder`

exec

```
@Deprecated(since="18")
public Process exec(String command,
                    String[] envp)
            throws IOException
```

Deprecated.
This method is error-prone and should not be used, the corresponding method `exec(String[], String[])` or `ProcessBuilder` should be used instead. The command string is broken into tokens using only whitespace characters. For an argument with an embedded space, such as a filename, this can cause problems as the token does not include the full filename.

Executes the specified string command in a separate process with the specified environment.

This is a convenience method. An invocation of the form `exec(command, envp)` behaves in exactly the same way as the invocation `exec(command, envp, null)`.

Implementation Note:

In the reference implementation, logging of the created process can be enabled, see `ProcessBuilder.start()` for details.

Parameters:

`command` - a specified system command.

`envp` - array of strings, each element of which has environment variable settings in the format *name=value*, or `null` if the subprocess should inherit the environment of the current process.

Returns:

A new `Process` object for managing the subprocess

Throws:

`IOException` - If an I/O error occurs

`NullPointerException` - If `command` is `null`, or one of the elements of `envp` is `null`

`IllegalArgumentException` - If `command` is empty

See Also:

`exec(String[], String[], File)`,
`ProcessBuilder`

exec

```
@Deprecated(since="18")
public Process exec(String command,
                    String[] envp,
                    File dir)
             throws IOException
```

Deprecated.
This method is error-prone and should not be used, the corresponding method `exec(String[], String[], File)` or `ProcessBuilder` should be used instead. The command string is broken into tokens using only whitespace characters. For an argument with an embedded space, such as a filename, this can cause problems as the token does not include the full filename.

Executes the specified string command in a separate process with the specified environment and working directory.

This is a convenience method. An invocation of the form `exec(command, envp, dir)` behaves in exactly the same way as the invocation `exec(cmdarray, envp, dir)`, where `cmdarray` is an array of all the tokens in `command`.

More precisely, the `command` string is broken into tokens using a `StringTokenizer` created by the call `new StringTokenizer(command)` with no further modification of the character categories. The tokens produced by the tokenizer are then placed in the new string array `cmdarray`, in the same order.

Implementation Note:

In the reference implementation, logging of the created process can be enabled, see `ProcessBuilder.start()` for details.

Parameters:

`command` - a specified system command.

`envp` - array of strings, each element of which has environment variable settings in the format *name=value*, or `null` if the subprocess should inherit the environment of the current process.

`dir` - the working directory of the subprocess, or `null` if the subprocess should inherit the working directory of the current process.

Returns:

A new `Process` object for managing the subprocess

Throws:

`IOException` - If an I/O error occurs

`NullPointerException` - If `command` is `null`, or one of the elements of `envp` is `null`

`IllegalArgumentException` - If `command` is empty

Since:

1.3

See Also:

`ProcessBuilder`

exec

```
public Process exec(String[] cmdarray)
             throws IOException
```

Executes the specified command and arguments in a separate process.

This is a convenience method. An invocation of the form `exec(cmdarray)` behaves in exactly the same way as the invocation `exec(cmdarray, null, null)`.

Implementation Note:

In the reference implementation, logging of the created process can be enabled, see `ProcessBuilder.start()` for details.

Parameters:

`cmdarray` - array containing the command to call and its arguments.

Returns:

A new `Process` object for managing the subprocess

Throws:

`IOException` - If an I/O error occurs

`NullPointerException` - If `cmdarray` is `null`, or one of the elements of `cmdarray` is `null`

`IndexOutOfBoundsException` - If `cmdarray` is an empty array (has length `0`)

See Also:

`ProcessBuilder`

exec

```
public Process exec(String[] cmdarray,
                    String[] envp)
             throws IOException
```

Executes the specified command and arguments in a separate process with the specified environment.

This is a convenience method. An invocation of the form `exec(cmdarray, envp)` behaves in exactly the same way as the invocation `exec(cmdarray, envp, null)`.

Implementation Note:

In the reference implementation, logging of the created process can be enabled, see `ProcessBuilder.start()` for details.

Parameters:

`cmdarray` - array containing the command to call and its arguments.

`envp` - array of strings, each element of which has environment variable settings in the format *name*=*value*, or `null` if the subprocess should inherit the environment of the current process.

Returns:

A new `Process` object for managing the subprocess

Throws:

`IOException` - If an I/O error occurs

`NullPointerException` - If `cmdarray` is `null`, or one of the elements of `cmdarray` is `null`, or one of the elements of `envp` is `null`

`IndexOutOfBoundsException` - If `cmdarray` is an empty array (has length `0`)

See Also:

`ProcessBuilder`

exec

```
public Process exec(String[] cmdarray,
                    String[] envp,
                    File dir)
             throws IOException
```

Executes the specified command and arguments in a separate process with the specified environment and working directory.

Given an array of strings `cmdarray`, representing the tokens of a command line, and an array of strings `envp`, representing "environment" variable settings, this method creates a new process in which to execute the specified command.

This method checks that `cmdarray` is a valid operating system command. Which commands are valid is system-dependent, but at the very least the command must be a non-empty list of non-null strings.

If `envp` is `null`, the subprocess inherits the environment settings of the current process.

A minimal set of system dependent environment variables may be required to start a process on some operating systems. As a result, the subprocess may inherit additional environment variable settings beyond those in the specified environment. The minimal set of system dependent environment variables may override the values provided in the environment.

`ProcessBuilder.start()` is now the preferred way to start a process with a modified environment.

The working directory of the new subprocess is specified by `dir`. If `dir` is `null`, the subprocess inherits the current working directory of the current process.

Starting an operating system process is highly system-dependent. Among the many things that can go wrong are:

- The operating system program file was not found.
- Access to the program file was denied.
- The working directory does not exist.

In such cases an exception will be thrown. The exact nature of the exception is system-dependent, but it will always be a subclass of `IOException`.

If the operating system does not support the creation of processes, an `UnsupportedOperationException` will be thrown.

Implementation Note:

In the reference implementation, logging of the created process can be enabled, see `ProcessBuilder.start()` for details.

Parameters:

`cmdarray` - array containing the command to call and its arguments.

`envp` - array of strings, each element of which has environment variable settings in the format *name*=*value*, or `null` if the subprocess should inherit the environment of the current process.

`dir` - the working directory of the subprocess, or `null` if the subprocess should inherit the working directory of the current process.

Returns:

A new `Process` object for managing the subprocess

Throws:

`UnsupportedOperationException` - If the operating system does not support the creation of processes.

`IOException` - If an I/O error occurs

`NullPointerException` - If `cmdarray` is `null`, or one of the elements of `cmdarray` is `null`, or one of the elements of `envp` is `null`

`IndexOutOfBoundsException` - If `cmdarray` is an empty array (has length `0`)

Since:

1.3

See Also:

`ProcessBuilder`

availableProcessors

```
public int availableProcessors()
```

Returns the number of processors available to the Java virtual machine.

This value may change during a particular invocation of the virtual machine. Applications that are sensitive to the number of available processors should therefore occasionally poll this property and adjust their resource usage appropriately.

Returns:

the maximum number of processors available to the virtual machine; never smaller than one

Since:

1.4

freeMemory

```
public long freeMemory()
```

Returns the amount of free memory in the Java Virtual Machine. Calling the `gc` method may result in increasing the value returned by `freeMemory.`

Returns:

an approximation to the total amount of memory currently available for future allocated objects, measured in bytes.

totalMemory

```
public long totalMemory()
```

Returns the total amount of memory in the Java virtual machine. The value returned by this method may vary over time, depending on the host environment.

Note that the amount of memory required to hold an object of any given type may be implementation-dependent.

Returns:

the total amount of memory currently available for current and future objects, measured in bytes.

maxMemory

```
public long maxMemory()
```

Returns the maximum amount of memory that the Java virtual machine will attempt to use. If there is no inherent limit then the value `Long.MAX_VALUE` will be returned.

Returns:

the maximum amount of memory that the virtual machine will attempt to use, measured in bytes

Since:

1.4

gc

```
public void gc()
```

Runs the garbage collector in the Java Virtual Machine.

Calling this method suggests that the Java Virtual Machine expend effort toward recycling unused objects in order to make the memory they currently occupy available for reuse by the Java Virtual Machine. When control returns from the method call, the Java Virtual Machine has made a best effort to reclaim space from all unused objects. There is no guarantee that this effort will recycle any particular number of unused objects, reclaim any particular amount of space, or complete at any particular time, if at all, before the method returns or ever. There is also no guarantee that this effort will determine the change of reachability in any particular number of objects, or that any particular number of `Reference` objects will be cleared and enqueued.

The name `gc` stands for "garbage collector". The Java Virtual Machine performs this recycling process automatically as needed, in a separate thread, even if the `gc` method is not invoked explicitly.

The method `System.gc()` is the conventional and convenient means of invoking this method.

runFinalization

```
@Deprecated(since="18",
            forRemoval=true)
public void runFinalization()
```

Deprecated, for removal: This API element is subject to removal in a future version.
Finalization has been deprecated for removal. See `Object.finalize()` for background information and details about migration options.

When running in a JVM in which finalization has been disabled or removed, no objects will be pending finalization, so this method does nothing.

Runs the finalization methods of any objects pending finalization. Calling this method suggests that the Java virtual machine expend effort toward running the `finalize` methods of objects that have been found to be discarded but whose `finalize` methods have not yet been run. When control returns from the method call, the virtual machine has made a best effort to complete all outstanding finalizations.

The virtual machine performs the finalization process automatically as needed, in a separate thread, if the `runFinalization` method is not invoked explicitly.

The method `System.runFinalization()` is the conventional and convenient means of invoking this method.

See *Java Language Specification*:

12.6 Finalization of Class Instances

See Also:

`Object.finalize()`

load

```
public void load(String filename)
```

`load` is a restricted method of the Java platform.
Programs can only use `load` when access to restricted methods is enabled. Restricted methods are unsafe, and, if used incorrectly, might crash the JVM or result in memory corruption.

Loads the native library specified by the filename argument. The filename argument must be an absolute path name. (for example `Runtime.getRuntime().load("/home/avh/lib/libX11.so");`). If the filename argument, when stripped of any platform-specific library prefix, path, and file extension, indicates a library whose name is, for example, L, and a native library called L is statically linked with the VM, then the JNI_OnLoad_L function exported by the library is invoked rather than attempting to load a dynamic library. A filename matching the argument does not have to exist in the file system. See the JNI Specification for more details. Otherwise, the

filename argument is mapped to a native library image in an implementation-dependent manner.

This is similar to the method `loadLibrary(String)` RESTRICTED, but it accepts a general file name as an argument rather than just a library name, allowing any file of native code to be loaded.

The method `System.load(String)` RESTRICTED is the conventional and convenient means of invoking this method.

Parameters:

`filename` - the file to load.

Throws:

`UnsatisfiedLinkError` - if either the filename is not an absolute path name, the native library is not statically linked with the VM, or the library cannot be mapped to a native library image by the host system.

`NullPointerException` - if `filename` is `null`

`IllegalCallerException` - if the caller is in a module that does not have native access enabled.

External Specifications

Java Native Interface Specification

See Also:

`getRuntime()`

loadLibrary

```
public void loadLibrary(String libname)
```

`loadLibrary` is a restricted method of the Java platform.
Programs can only use `loadLibrary` when access to restricted methods is enabled.
Restricted methods are unsafe, and, if used incorrectly, might crash the JVM or result in memory corruption.

Loads the native library specified by the `libname` argument. The `libname` argument must not contain any platform specific prefix, file extension or path. If a native library called `libname` is statically linked with the VM, then the JNI_OnLoad_`libname` function exported by the library is invoked. See the JNI Specification for more details. Otherwise, the libname argument is loaded from a system library location and mapped to a native library image in an implementation-dependent manner.

The method `System.loadLibrary(String)` RESTRICTED is the conventional and convenient means of invoking this method. If native methods are to be used in

the implementation of a class, a standard strategy is to put the native code in a library file (call it `LibFile`) and then to put a static initializer:

```
static { System.loadLibrary("LibFile"); }
```

within the class declaration. When the class is loaded and initialized, the necessary native code implementation for the native methods will then be loaded as well.

If this method is called more than once with the same library name, the second and subsequent calls are ignored.

Parameters:

`libname` - the name of the library.

Throws:

`UnsatisfiedLinkError` - if either the libname argument contains a file path, the native library is not statically linked with the VM, or the library cannot be mapped to a native library image by the host system.

`NullPointerException` - if `libname` is `null`

`IllegalCallerException` - if the caller is in a module that does not have native access enabled.

External Specifications

Java Native Interface Specification

version

`public static Runtime.Version version()`

Returns the version of the Java Runtime Environment as a `Runtime.Version`.

Returns:

the `Runtime.Version` of the Java Runtime Environment

Since:

9

Report a bug or suggest an enhancement
For further API reference and developer documentation see the Java SE Documentation, which contains more detailed, developer-targeted descriptions with conceptual overviews, definitions of terms, workarounds, and working code examples. Other versions.
Java is a trademark or registered trademark of Oracle and/or its affiliates in the US and other countries.
Copyright © 1993, 2026, Oracle and/or its affiliates, 500 Oracle Parkway, Redwood Shores, CA 94065 USA.
All rights reserved. Use is subject to license terms and the documentation redistribution policy.

Scripting on this page tracks web page traffic, but does not change the content in any way.